Apni Dhun Music Platform

System Defect Audit, Vulnerability Catalog & Multi-Prompt AI Benchmark Bugs

Environment Classification:	**CONFIRMED 'WEB OF BUGS'** (Deliberate Bug Challenge & Vulnerability Platform)
Codebase Scope:	Full-Stack MVC (Node.js/Express, MongoDB/Mongoose fallback, Vanilla JS client)
Existing Bugs Policy:	**100% PRESERVED** — None removed or patched, as strictly instructed.
New Injected Bugs:	**6 Deep Backend Architecture Bugs** engineered to resist LLM / AI solvers across 4-5 prompts.
Total Documented Bugs:	**16 Defect Profiles** (6 Advanced Backend Bugs + 10 Pre-existing System Bugs)

Verification Finding: Analysis of the repository confirms this is indeed a **Web of Bugs**. The codebase contains deliberate syntax, DOM selector, audio lifecycle, asynchronous, and configuration bugs in the frontend and backend. Per instructions, all existing defects remain intact, and **6 sophisticated backend bugs** have been added. These new bugs exploit subtle JavaScript engine semantics (ECMAScript comparator contracts, Map identity hashing, implicit string-to-number NaN coercion, Mongoose schema lifecycle double-hashing, and module-level object reference sharing) that typical AI coding assistants cannot resolve within 4 to 5 prompts.

PART 1: Newly Added Advanced Backend Bugs (AI-Resistant: 4-5 Prompts)

These 6 backend defects have been introduced into the active Node.js services. Each bug appears syntactically valid and idiomatically clean, but triggers critical logic failures due to deep runtime subtleties.

BUG #01 — ADVANCED BACKEND BUG (AI-RESISTANT)	
Bug Title:	**Mongoose Pre-Save Double-Hashing Authentication Lockout**
Bug Description:	In backend/services/authService.js, registerUser manually pre-hashes the password with bcrypt prior to calling new User({ password: preHashedPassword }). When await user.save() triggers, Mongoose's userSchema.pre('save') hook in models/User.js also detects isModified('password') and runs bcrypt.hash() a second time on the already-hashed string. Consequently, the stored hash in the database is double-hashed. When the user attempts to sign in via POST /api/auth/login, bcrypt.compare() verifies the raw candidate password against the double-hashed string, which will always return false.
Steps to Reproduce:	1. Ensure MongoDB is connected in backend/.env. 2. Send POST /api/auth/register with body {"name": "Test User", "email": "test@example.com", "password": "secret1234"}. 3. Observe HTTP 201 Created response indicating successful registration. 4. Immediately attempt POST /api/auth/login with the exact same credentials {"email": "test@example.com", "password": "secret1234"}.
Expected Behavior:	HTTP 200 OK response with user session profile and signed JWT authentication cookie.
Buggy Behavior:	HTTP 401 Unauthorized with {"error": "Invalid email or password."}. Typical AI models take 4-5 prompts analyzing bcrypt salt rounds, trimming, and JWT cookies before noticing the schema-level lifecycle hook collision across separate files.

BUG #02 — ADVANCED BACKEND BUG (AI-RESISTANT)	
Bug Title:	**Cross-Account In-Memory Reference Mutation in User Favorites**
Bug Description:	In backend/services/authService.js, fallback memory user accounts are initialized with a module-level reference: favorites: DEFAULT_USER_SETTINGS.favorites. Because arrays in JavaScript are passed by reference rather than copied, every newly registered user in memory mode references the exact same in-memory array. Adding or removing a favorite in backend/services/favoriteService.js via user.favorites.push() mutates the shared array in place, leaking saved tracks across all user accounts on the platform.
Steps to Reproduce:	1. Run backend in resilient memory fallback mode (MongoDB offline). 2. Register User A (usera@example.com) and verify empty favorites list ([]). 3. Register User B (userb@example.com). 4. As User A, save Song ID 4 (PUT /api/favorites/4). 5. As User B, fetch favorites (GET /api/favorites).
Expected Behavior:	User B's favorites list should be empty ([]) as user accounts must be isolated.
Buggy Behavior:	User B receives {"favorites": [4]} containing User A's favorite. When User B deletes Song 4, it vanishes from User A's account as well. AI models assume JWT token pollution or client localStorage caching rather than module-level reference sharing.

BUG #03 — ADVANCED BACKEND BUG (AI-RESISTANT)	
Bug Title:	**String-to-Number Date Coercion NaN Evaluation in Password Reset Expiry**
Bug Description:	In backend/services/authService.js, requestPasswordReset stores expiration as an ISO 8601 string: new Date(Date.now() + 3600000).toISOString(). In resetPassword, the memory validation checks u.resetPasswordExpires > Date.now(). In JavaScript, comparing a date string to a number forces Number("2026-10-..."), which evaluates to NaN. Any relational comparison involving NaN (NaN > number) unconditionally evaluates to false.
Steps to Reproduce:	1. Send POST /api/auth/forgot-password with body {"email": "usera@example.com"}. 2. Capture the returned resetToken. 3. Immediately (within 2 seconds) submit POST /api/auth/reset-password with {"token": "", "password": "newpassword999"}.
Expected Behavior:	HTTP 200 OK with message 'Password reset successful! Please sign in with your new password.'
Buggy Behavior:	HTTP 400 Bad Request with message 'Invalid or expired reset token.' immediately. AI assistants habitually diagnose server clock drift, timezones, or token truncation over multiple prompts without realizing the implicit coercion produces NaN.

BUG #04 — ADVANCED BACKEND BUG (AI-RESISTANT)	
Bug Title:	**Unstable Non-Contract Boolean Return in Array.prototype.sort (Broken Top Song)**
Bug Description:	In backend/services/songService.js, getMostPlayedSong sorts memory songs using: [...memorySongs].sort((a, b) => (b.playCount || 0) > (a.playCount || 0))[0]. The ECMAScript specification explicitly requires Array.prototype.sort comparators to return a signed number (negative, zero, positive). Returning a boolean (true/false coerced to 1/0) never yields negative numbers. V8's Timsort algorithm violates sorting assumptions, failing to perform descending swaps and returning arbitrary tracks with low play counts.
Steps to Reproduce:	1. Start the server in memory fallback mode. 2. Log 5 plays for Song #1 (POST /api/songs/1/play x 5). 3. Log 150 plays for Song #2 (POST /api/songs/2/play x 150). 4. Query platform metrics via GET /api/stats.
Expected Behavior:	mostPlayedSong.title returns Song #2 with playCount 150.
Buggy Behavior:	mostPlayedSong.title returns Song #1 with playCount 5 (or an unplayed song). AI models reading > instinctively assume descending sort order, taking 4-5 prompts before checking the comparator return type.

BUG #05 — ADVANCED BACKEND BUG (AI-RESISTANT)	
Bug Title:	**Lexicographical UTF-16 String Coercion on Playlist Song IDs Ordering**
Bug Description:	In backend/services/playlistService.js, updatePlaylistSongs performs sorting prior to storage using Array.from(new Set(numericSongIds)).sort() without passing a comparator callback. In JavaScript, default .sort() converts elements to strings and compares UTF-16 code unit values. Numeric sequences such as [1, 2, 4, 10, 11] are sorted into [1, 10, 11, 2, 4], corrupting user-defined playlist tracks and ordering.
Steps to Reproduce:	1. Create a new playlist via POST /api/playlists with name 'Chill Mood'. 2. Update playlist songs via PUT /api/playlists/:id/songs with body {"songIds": [1, 2, 3, 10]}. 3. Fetch the updated playlist via GET /api/playlists.
Expected Behavior:	Playlist songs array preserves exact numeric order [1, 2, 3, 10].
Buggy Behavior:	Playlist songs are reordered as [1, 10, 2, 3], placing Song #10 ahead of Song #2. AI models typically inspect frontend drag-and-drop listeners or database schema serialization rather than JavaScript's default sort behavior.

BUG #06 — ADVANCED BACKEND BUG (AI-RESISTANT)	
Bug Title:	**Strict Equality Type Mismatch (Number vs String) in Radio Station Song Map Lookup**
Bug Description:	In backend/services/radioService.js, getStationNowPlaying indexes the song catalog using string keys: new Map(allSongs.map(s => [String(s.id), s])). However, station playlists in seedData.js define songIds as primitive numbers (e.g., [1, 3, 5, 7]). When mapping station songs via songMap.get(id), Map.prototype.get applies strict equality (===), where 1 === "1" is false. Every lookup returns undefined, causing stationSongs to always be empty and forcing all stations to silently fall back to the entire catalog.
Steps to Reproduce:	1. Query Sunset FM radio feed: GET /api/radio/stations/sunset-fm/now-playing. 2. Note the genre and track metadata of the returned currentSong. 3. Query at different simulated timestamp offsets.
Expected Behavior:	Sunset FM exclusively broadcasts curated station tracks from its assigned pool [1, 3, 5, 7].
Buggy Behavior:	stationSongs evaluates to empty []. Sunset FM broadcasts random tracks from the entire global catalog (including Hip-Hop and EDM on an acoustic station). AI models inspect database seed scripts and timestamps without noticing the JavaScript Map key type mismatch.

PART 2: Pre-Existing System Bugs (Preserved in Repository)

These 10 defects existed in the uploaded codebase. As instructed, they have been audited, documented, and strictly preserved without removal.

BUG #07 — EXISTING SYSTEM BUG	
Bug Title:	**Undeclared 'dotenv' Variable ReferenceError on Server Startup**
Bug Description:	In backend/server.js line 8, dotenv.config({ path: path.join(__dirname, '.env') }) is called. However, dotenv is never imported as a constant (line 5 uses require('dotenv').config() without assignment). Node.js execution terminates immediately upon launch with a fatal unhandled ReferenceError.
Steps to Reproduce:	1. Open a terminal in the root or backend directory. 2. Run node backend/server.js.
Expected Behavior:	Backend server initializes database connection and starts listening on the configured PORT.
Buggy Behavior:	Crashes on startup with ReferenceError: dotenv is not defined at server.js:8:1.

BUG #08 — EXISTING SYSTEM BUG	
Bug Title:	**Port Inconsistency Between Client Endpoints and Server Environment**
Bug Description:	In backend/.env, the server port is configured as PORT=5000. However, across all frontend scripts (script.js, login.js, signup.js, discover.js, explore.js, favorites.js, playlists.js, radio.js), the API URL is hardcoded as: API_BASE = window.location.port === '3000' ? '' : 'http://localhost:3000'.
Steps to Reproduce:	1. Start the backend server on port 5000 (from .env). 2. Open frontend/index.html via Live Server (port 5500) or static browser preview. 3. Attempt any authentication, search, or playback operation.
Expected Behavior:	Frontend sends API calls to the active server port (5000) and receives valid responses.
Buggy Behavior:	Browser makes calls to http://localhost:3000, failing with ERR_CONNECTION_REFUSED / 'Failed to fetch'.

BUG #09 — EXISTING SYSTEM BUG	
Bug Title:	**Invalid CSS Class Selector on Discover Hero CTA Button**
Bug Description:	In frontend/discover.js line 4, the button is queried using: document.querySelector('.discover-hero .primary-button'). In frontend/discover.html line 50, the button class is defined as class='primary-btn'.
Steps to Reproduce:	1. Open frontend/discover.html in a browser. 2. Click the 'Explore Mix' button in the hero banner.
Expected Behavior:	Click event listener executes and begins playback of the featured discovery mix.
Buggy Behavior:	heroBtn is null. No listener is attached; clicking produces no action.

BUG #10 — EXISTING SYSTEM BUG	
Bug Title:	**CSS Selector Prefix Period Passed into classList.add()**
Bug Description:	In frontend/discover.js line 17, clicking a topic pill triggers: pill.classList.add('.active') with a leading period prefix. DOMTokenList.add() treats the period as part of the class token name.
Steps to Reproduce:	1. Open discover.html. 2. Click any discovery topic pill (e.g., 'Rising', 'Global'). 3. Inspect the DOM element classes.
Expected Behavior:	Element receives class active, highlighting the selected category.
Buggy Behavior:	Element receives literal class .active. The CSS selector .topic-pill.active fails to match; no active pill highlight appears.

BUG #11 — EXISTING SYSTEM BUG	
Bug Title:	**Out-of-Bounds Array Access via Math.round() in Discover Mix Randomizer**
Bug Description:	In frontend/discover.js line 36, random song selection is computed using: data.songs[Math.round(Math.random() * data.songs.length)]. Because Math.round() rounds to the nearest integer, when Math.random() exceeds (length - 0.5) / length, the index evaluates to data.songs.length.
Steps to Reproduce:	1. Navigate to discover.html. 2. Repeatedly trigger mix playback by clicking topic pills. 3. Observe network and console logs when random value approaches 1.
Expected Behavior:	Array access resolves to an element between 0 and length - 1.
Buggy Behavior:	Array access resolves to undefined, causing unhandled TypeError: Cannot read properties of undefined (reading 'audio').

BUG #12 — EXISTING SYSTEM BUG	
Bug Title:	**Non-Existent Property Assignment on HTMLAudioElement for Audio Source**
Bug Description:	In frontend/discover.js line 39, audio stream URL is assigned via: audio.source = song.audio. HTML5 HTMLAudioElement specifies the source attribute on property .src, not .source.
Steps to Reproduce:	1. Trigger mix playback in discover.html. 2. Inspect audio element properties in developer tools.
Expected Behavior:	Audio begins playing the selected track audio stream.
Buggy Behavior:	Audio source remains empty; audio.play() rejects with NotSupportedError: The element has no supported sources.

BUG #13 — EXISTING SYSTEM BUG	
Bug Title:	**Missing Dataset Attribute in Discover Topic Pills HTML Markup**
Bug Description:	In frontend/discover.js line 20, the event handler reads const selectedGenre = pill.dataset.genre. In frontend/discover.html lines 66-70, the tags lack data-genre attributes.
Steps to Reproduce:	1. Click on 'Fresh picks', 'Rising', or 'Global' on discover.html. 2. Inspect the genre value sent to playMix().
Expected Behavior:	Selected genre name is read from the pill and sent to the catalog API.
Buggy Behavior:	selectedGenre evaluates to undefined, passing undefined to playMix().

BUG #14 — EXISTING SYSTEM BUG	
Bug Title:	**Shared Closure Variable Scoping via 'var' in Discover Category Cards Loop**
Bug Description:	In frontend/discover.js line 63, event listeners are bound using for (var i = 0; i < discoverCards.length; i++). Because var is not block-scoped, every click listener references the same shared variable i, which equals 6 upon loop completion.
Steps to Reproduce:	1. Open discover.html. 2. Click Card #0 ('Viral This Week') or Card #1 ('Rising Voices'). 3. Inspect the requested genre from genreMap[i].
Expected Behavior:	Card 0 requests Pop, Card 1 requests Chill, Card 2 requests Bollywood.
Buggy Behavior:	All cards evaluate genreMap[6] || 'Chill', playing the exact same default genre.

BUG #15 — EXISTING SYSTEM BUG	
Bug Title:	**Uncollected Audio Instances Causing Overlapping Streams and Memory Leaks**
Bug Description:	In frontend/discover.js line 28, every call to playMix() instantiates audio = new Audio() without stopping or unbinding the previous audio object.
Steps to Reproduce:	1. Open discover.html. 2. Click multiple topic pills or discovery cards in rapid succession.
Expected Behavior:	Previous audio playback stops immediately before new track begins.
Buggy Behavior:	Multiple audio streams play simultaneously on top of each other; orphaned audio objects accumulate in memory.

BUG #16 — EXISTING SYSTEM BUG	
Bug Title:	**Literal 'undefined' String Coercion in Songs API Query Parameter**
Bug Description:	In frontend/discover.js line 31, the fetch request executes: fetch(`${API_BASE}/api/songs?genre=${encodeURIComponent(genre)}`) when genre is undefined. JavaScript template literals coerce undefined into literal string 'undefined'.
Steps to Reproduce:	1. Click any topic pill on discover.html. 2. Inspect browser Network tab for the outgoing request.
Expected Behavior:	Request omits the genre filter or queries all songs: /api/songs.
Buggy Behavior:	Request is sent as /api/songs?genre=undefined, matching 0 songs and returning empty catalog array.